Die Schnorr-Identifikation

Mathematische Grundlagen, Protokoll und praktische Einordnung

Fabian Grebert

13. November 2026

Schreibhinweis: Zum Schluss schreiben: Problem, Vorgehen und wichtigste Ergebnisse in einem kurzen Absatz zusammenfassen. Keine neuen Argumente einführen.

Inhaltsverzeichnis

1 Einleitung und Problemstellung **2**

2 Mathematische Grundlagen **2**
2.1 Gruppen und zyklische Gruppen . . . 2
2.2 Modulare Arithmetik und Untergruppen . . . 3
2.3 Das diskrete Logarithmusproblem . . . 5
2.4 Verwendete Notation . . . 6

3 Die Schnorr-Identifikation **6**
3.1 Parameter und Schlüsselerzeugung . . . 7
3.2 Ablauf des Drei-Schritt-Protokolls . . . 7
3.3 Korrektheit . . . 9
3.4 Numerisches Beispiel . . . 10
3.5 Sicherheitsidee und Grenzen . . . 12
3.6 Abgrenzung zu nichtinteraktiven Verfahren . . . 17
3.7 Zusammenfassung . . . 17

4 Praxisbezug und Aufwandsabschätzung **18**
4.1 Optional: Ein kleines Rechenbeispiel . . . 18
4.2 Aufwand eines Angriffs . . . 18
4.3 Anwendungsfälle und verwandte Verfahren . . . 18

5 Vor- und Nachteile sowie Einsatzbedingungen **18**
5.1 Vorteile . . . 18
5.2 Nachteile und Voraussetzungen . . . 18
5.3 Wann lohnt sich das Verfahren? . . . 18

6 Fazit **19**

Literaturverzeichnis **19**

1 Einleitung und Problemstellung

Schreibhinweis: Erklären, welches Problem gelöst wird: Eine Person soll die Kenntnis eines geheimen Schlüssels nachweisen, ohne diesen offenzulegen. Rollen von beweisender und prüfender Partei einführen und die vertrauenswürdige Zuordnung eines öffentlichen Schlüssels zur Identität ansprechen.

Schreibhinweis: Schnorr-Identifikation historisch einordnen und von Verschlüsselung und digitalen Signaturen abgrenzen. Leitfrage und Aufbau des Papers nennen. Ausgangsquelle: [1].

2 Mathematische Grundlagen

Das Schnorr-Identifikationsprotokoll basiert auf algebraischen Strukturen und der angenommenen Schwierigkeit des diskreten Logarithmusproblems. Für das Verständnis des Protokolls werden im Folgenden die notwendigen mathematischen Grundlagen eingeführt. Hierzu zählen insbesondere Gruppen, zyklische Gruppen, modulare Arithmetik sowie das diskrete Logarithmusproblem.

2.1 Gruppen und zyklische Gruppen

Eine Gruppe ist eine algebraische Struktur, die aus einer Menge und einer darauf definierten binären Verknüpfung besteht. Gruppen bilden eine wesentliche Grundlage moderner asymmetrischer kryptografischer Verfahren.

Definition 2.1 (Gruppe)**.** *Eine Gruppe $(G, \circ)$ besteht aus einer nichtleeren Menge G und einer binären Verknüpfung*

$$\circ\colon G \times G \to G,$$

für die folgende Eigenschaften gelten:

1. ***Abgeschlossenheit:*** *Für alle $a, b \in G$ gilt*

$$a \circ b \in G.$$

2. ***Assoziativität:*** *Für alle $a, b, c \in G$ gilt*

$$(a \circ b) \circ c = a \circ (b \circ c).$$

3. ***Neutrales Element:*** *Es existiert ein Element $e \in G$, sodass für alle $a \in G$ gilt*

$$a \circ e = e \circ a = a.$$

4. ***Inverse Elemente:*** *Zu jedem Element $a \in G$ existiert ein inverses Element $a^{-1} \in G$, sodass*

$$a \circ a^{-1} = a^{-1} \circ a = e.$$

Gilt zusätzlich für alle $a, b \in G$

$$a \circ b = b \circ a,$$

so wird die Gruppe als kommutative oder abelsche Gruppe bezeichnet.

Ein einfaches Beispiel ist die Menge der ganzen Zahlen $\mathbb{Z}$ mit der Addition. Die Summe zweier ganzer Zahlen ist erneut eine ganze Zahl, die Addition ist assoziativ, das neutrale Element ist 0 und das inverse Element einer Zahl a ist $-a$. Somit bildet $(\mathbb{Z}, +)$ eine abelsche Gruppe.

Für kryptografische Anwendungen sind insbesondere endliche zyklische Gruppen von Bedeutung.

Definition 2.2 (Zyklische Gruppe)**.** *Eine Gruppe $\mathbb{G}$ heißt zyklisch, wenn ein Element $g \in \mathbb{G}$ existiert, durch dessen wiederholte Verknüpfung sämtliche Elemente der Gruppe erzeugt werden können.*

In multiplikativer Schreibweise gilt

$$\mathbb{G} = \langle g \rangle = \{g^k \mid k \in \mathbb{Z}\}.$$

Das Element g wird als Generator oder erzeugendes Element der Gruppe bezeichnet.

Die Anzahl der Elemente einer endlichen Gruppe wird als Gruppenordnung bezeichnet. Besitzt eine zyklische Gruppe $\mathbb{G}$ die Ordnung q, so gilt

$$|\mathbb{G}| = q$$

und

$$\mathbb{G} = \{e, g, g^2, \ldots, g^{q-1}\}.$$

Für einen Generator g einer Gruppe der Ordnung q gilt insbesondere

$$g^q = e.$$

Dabei ist q die kleinste positive ganze Zahl, für die diese Gleichung erfüllt ist. Die Ordnung eines Generators entspricht somit der Ordnung der gesamten Gruppe.

Jede zyklische Gruppe ist abelsch. Besitzt eine zyklische Gruppe eine prime Ordnung q, so ist jedes vom neutralen Element verschiedene Gruppenelement ebenfalls ein Generator der Gruppe.

2.2 Modulare Arithmetik und Untergruppen

Die modulare Arithmetik beschäftigt sich mit der Berechnung von Restklassen bezüglich einer positiven ganzen Zahl n, die als Modulus bezeichnet wird.

Zwei ganze Zahlen a und b heißen kongruent modulo n, wenn ihre Differenz durch n teilbar ist:

$$a \equiv b \pmod{n} \quad \Longleftrightarrow \quad n \mid (a - b).$$

Beispielsweise gilt

$$17 \equiv 2 \pmod{5},$$

da $17 - 2 = 15$ durch 5 teilbar ist.

Die Menge der Restklassen modulo n wird mit $\mathbb{Z}_n$ bezeichnet. Sie kann durch die Repräsentanten

$$\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$$

dargestellt werden.

Addition und Multiplikation werden dabei modulo n ausgeführt. Beispielsweise gilt in $\mathbb{Z}_5$

$$3 + 4 \equiv 2 \pmod{5}$$

sowie

$$3 \cdot 4 \equiv 2 \pmod{5}.$$

Für eine Primzahl p bilden die von null verschiedenen Restklassen modulo p bezüglich der modularen Multiplikation eine Gruppe:

$$\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}.$$

Diese multiplikative Gruppe besitzt die Ordnung $p - 1$ und ist zyklisch. Jedes ihrer Elemente besitzt ein multiplikatives Inverses modulo p.

Für kryptografische Verfahren wie das Schnorr-Identifikationsprotokoll wird häufig eine zyklische Untergruppe primer Ordnung verwendet.

Definition 2.3 (Untergruppe)**.** *Eine nichtleere Teilmenge $H \subseteq G$ einer Gruppe $(G, \circ)$ heißt Untergruppe, wenn sie bezüglich derselben Verknüpfung selbst eine Gruppe bildet.*

Seien p und q Primzahlen, für die

$$q \mid (p-1)$$

gilt. Da die multiplikative Gruppe $\mathbb{Z}_p^*$ zyklisch ist, existiert eine Untergruppe $\mathbb{G} \subseteq \mathbb{Z}_p^*$ der Ordnung q.

Ein Generator g dieser Untergruppe kann beispielsweise aus einem Element $h \in \mathbb{Z}_p^*$ durch

$$g = h^{(p-1)/q} \bmod p$$

bestimmt werden, sofern $g \neq 1$ gilt.

Nach dem kleinen Satz von Fermat folgt

$$g^q \equiv h^{p-1} \equiv 1 \pmod{p}.$$

Da q prim ist und $g \neq 1$ gilt, besitzt g genau die Ordnung q. Damit erzeugt g die Untergruppe

$$\mathbb{G} = \langle g \rangle.$$

Beispiel. Zur Veranschaulichung werden die Primzahlen

$$p = 23, \qquad q = 11$$

gewählt. Es gilt

$$q \mid (p-1),$$

da 11 ein Teiler von 22 ist.

Mit $h = 5$ ergibt sich

$$g = 5^{22/11} \bmod 23 = 5^2 \bmod 23 = 2.$$

Das Element $g = 2$ besitzt modulo 23 die Ordnung 11, denn

$$2^{11} \equiv 1 \pmod{23}.$$

Die erzeugte Untergruppe lautet

$$\mathbb{G} = \{1, 2, 4, 8, 16, 9, 18, 13, 3, 6, 12\}.$$

Damit besitzt $\mathbb{G}$ genau elf Elemente.

Bei der Verwendung einer solchen Untergruppe muss zwischen zwei unterschiedlichen modularen Berechnungen unterschieden werden:

- Operationen auf Gruppenelementen werden modulo p berechnet.
- Operationen auf Exponenten werden modulo q berechnet.

Da $g^q \equiv 1 \pmod{p}$ gilt, können Exponenten modulo q reduziert werden, ohne das resultierende Gruppenelement zu verändern.

Für beliebige ganze Zahlen a und k gilt daher

$$g^{a+kq} \equiv g^a \pmod{p}.$$

Beispielsweise ergibt sich für die gewählten Parameter

$$2^{14} \equiv 2^{14 \bmod 11} \equiv 2^3 \equiv 8 \pmod{23}.$$

Die kleinen Parameter dieses Beispiels dienen ausschließlich der Veranschaulichung. Für einen kryptografisch sicheren Einsatz sind wesentlich größere und geeignet gewählte Parameter erforderlich.

2.3 Das diskrete Logarithmusproblem

Die Sicherheit des Schnorr-Identifikationsprotokolls beruht wesentlich auf der angenommenen Schwierigkeit des diskreten Logarithmusproblems.

Definition 2.4 (Diskretes Logarithmusproblem)**.** *Sei $\mathbb{G} = \langle g \rangle$ eine endliche zyklische Gruppe der Ordnung q mit Generator g.*

Für ein gegebenes Gruppenelement $y \in \mathbb{G}$ besteht das diskrete Logarithmusproblem (Discrete Logarithm Problem, DLP) darin, einen Exponenten $x \in \mathbb{Z}_q$ zu bestimmen, sodass

$$y = g^x$$

gilt.

Der Exponent x wird als diskreter Logarithmus von y zur Basis g bezeichnet:

$$x = \log_g(y).$$

In einer zyklischen Gruppe der Ordnung q ist dieser Exponent modulo q eindeutig bestimmt.

Im Fall einer multiplikativen Untergruppe von $\mathbb{Z}_p^*$ lautet die entsprechende Gleichung

$$y \equiv g^x \pmod{p}.$$

Während die Berechnung von y aus einem bekannten Exponenten x mithilfe effizienter Verfahren zur modularen Exponentiation möglich ist, gilt die Umkehrung für geeignet gewählte Gruppen und ausreichend große Parameter als rechnerisch schwierig.

Diese Eigenschaft ermöglicht die Verwendung der modularen Exponentiation als kryptografische Einwegfunktion unter der Annahme, dass das diskrete Logarithmusproblem in der verwendeten Gruppe praktisch nicht effizient lösbar ist.

Beispiel. Für die zuvor eingeführten Parameter

$$p = 23, \qquad q = 11, \qquad g = 2$$

wird der geheime Exponent

$$x = 7$$

gewählt.

Der zugehörige öffentliche Wert lässt sich durch modulare Exponentiation berechnen:

$$y = g^x \bmod p = 2^7 \bmod 23 = 13.$$

Das diskrete Logarithmusproblem besteht in diesem Fall darin, aus den öffentlich bekannten Werten $p = 23$, $q = 11$, $g = 2$ und $y = 13$ den Exponenten x zu bestimmen:

$$2^x \equiv 13 \pmod{23}.$$

Aufgrund der kleinen Gruppenordnung kann die Lösung durch systematisches Ausprobieren aller möglichen Exponenten leicht gefunden werden. Bei ausreichend großen Gruppen ist eine solche vollständige Suche hingegen praktisch nicht durchführbar.

Neben der vollständigen Suche existieren weitere Algorithmen zur Berechnung diskreter Logarithmen. Dazu zählen beispielsweise der Baby-Step-Giant-Step-Algorithmus und der Pollard-Rho-Algorithmus. Für bestimmte Gruppen existieren zudem spezialisierte Verfahren.

Die Schwierigkeit des diskreten Logarithmusproblems hängt daher nicht ausschließlich von der Gruppenordnung, sondern auch von der mathematischen Struktur der verwendeten Gruppe und den verfügbaren Lösungsalgorithmen ab.

Im Schnorr-Identifikationsprotokoll dient der Exponent x als geheimer Schlüssel, während $y = g^x$ den zugehörigen öffentlichen Schlüssel darstellt. Ein Teilnehmer kann mithilfe des Protokolls den Besitz des geheimen Schlüssels nachweisen, ohne diesen unmittelbar offenzulegen.

2.4 Verwendete Notation

Für die Beschreibung des Schnorr-Identifikationsprotokolls wird im weiteren Verlauf dieser Arbeit eine zyklische Gruppe $\mathbb{G} = \langle g \rangle$ primer Ordnung q verwendet.

Die nachfolgende Tabelle fasst die verwendeten Symbole und deren Bedeutung zusammen.

Symbol	Bedeutung
p	Primzahl als Modulus der Gruppenelemente
q	Prime Ordnung der zyklischen Gruppe
$\mathbb{G}$	Zyklische Gruppe der Ordnung q
g	Generator der Gruppe $\mathbb{G}$
e	Neutrales Element der Gruppe
$x \in \mathbb{Z}_q$	Geheimer Schlüssel
$y = g^x$	Öffentlicher Schlüssel
$r \in \mathbb{Z}_q$	Frischer Zufallswert (Nonce)
$t = g^r$	Commitment
$c \in \mathbb{Z}_q$	Challenge
$s \in \mathbb{Z}_q$	Response

Der geheime Schlüssel x wird vom beweisenden Teilnehmer (Prover) geheim gehalten. Der öffentliche Schlüssel y steht hingegen dem prüfenden Teilnehmer (Verifier) zur Verfügung.

Für jede Ausführung des Protokolls wird ein neuer, unabhängiger und kryptografisch sicherer Zufallswert r aus $\mathbb{Z}_q$ gewählt. Aus diesem Wert wird das Commitment $t = g^r$ berechnet.

Die Challenge c wird im interaktiven Protokoll durch den Verifier zufällig gewählt. Der Prover berechnet anschließend unter Verwendung seines geheimen Schlüssels x die Response s.

In dieser Arbeit wird folgende Vorzeichenkonvention verwendet:

$$y = g^x, \qquad t = g^r, \qquad s = (r + cx) \bmod q.$$

Die zugehörige Verifikationsgleichung lautet

$$g^s = t \cdot y^c.$$

Für die konkrete Realisierung in einer Untergruppe von $\mathbb{Z}_p^*$ ergibt sich

$$g^s \equiv t \cdot y^c \pmod{p}.$$

In der Literatur finden sich teilweise alternative Darstellungen des Schnorr-Protokolls, beispielsweise mit einer Subtraktion bei der Berechnung der Response oder einem invers definierten öffentlichen Schlüssel. Solche Darstellungen sind bei konsistenter Anpassung der Verifikationsgleichung mathematisch äquivalent.

Im weiteren Verlauf dieser Arbeit wird ausschließlich die oben eingeführte Vorzeichenkonvention verwendet, um Mehrdeutigkeiten zu vermeiden.

Sofern nicht ausdrücklich anders angegeben, werden Operationen auf Gruppenelementen modulo p und Operationen auf Exponenten modulo q durchgeführt.

3 Die Schnorr-Identifikation

Das Schnorr-Identifikationsprotokoll ist ein interaktives kryptografisches Verfahren, mit dem eine Partei die Kenntnis eines geheimen Schlüssels nachweisen kann, ohne diesen gegenüber einer anderen Partei offenzulegen.

Das Verfahren basiert auf dem in Abschnitt 2 beschriebenen diskreten Logarithmusproblem. Es handelt sich um ein sogenanntes Challenge-Response-Verfahren, bei dem die Authentifizierung durch den Austausch von drei Nachrichten erfolgt.

Die beweisende Partei wird im Folgenden als *Prover* und die prüfende Partei als *Verifier* bezeichnet.

3.1 Parameter und Schlüsselerzeugung

Die Grundlage des Schnorr-Identifikationsprotokolls bildet eine endliche zyklische Gruppe

$$\mathbb{G} = \langle g \rangle$$

mit primer Ordnung q und einem Generator g.

Für die Realisierung in einer multiplikativen Untergruppe von $\mathbb{Z}_p^*$ werden zwei Primzahlen p und q gewählt, für die

$$q \mid (p-1)$$

gilt. Der Generator g besitzt dabei die Ordnung q, sodass

$$g^q \equiv 1 \pmod{p} \qquad \text{und} \qquad g \neq 1$$

gelten.

Die Parameter p, q und g sind öffentlich bekannt und müssen nicht geheim gehalten werden. Sie sind jedoch so zu wählen, dass das diskrete Logarithmusproblem in der verwendeten Gruppe praktisch nicht effizient lösbar ist.

Schlüsselerzeugung. Der Prover erzeugt seinen geheimen Schlüssel x, indem er einen Wert gleichverteilt aus den von null verschiedenen Elementen von $\mathbb{Z}_q$ auswählt:

$$x \xleftarrow{\$} \mathbb{Z}_q \setminus \{0\}.$$

Anschließend berechnet er den zugehörigen öffentlichen Schlüssel

$$y = g^x.$$

Bei der konkreten Realisierung in einer Untergruppe von $\mathbb{Z}_p^*$ erfolgt die Berechnung durch

$$y = g^x \bmod p.$$

Damit besteht das Schlüsselpaar aus

$$\begin{aligned} x &\quad \text{(geheimer Schlüssel)}, \\ y &\quad \text{(öffentlicher Schlüssel)}. \end{aligned}$$

Der geheime Schlüssel verbleibt ausschließlich beim Prover. Der öffentliche Schlüssel wird dem Verifier zur Verfügung gestellt und kann beispielsweise während einer Registrierung einer bestimmten Benutzeridentität zugeordnet werden.

Diese Zuordnung muss vertrauenswürdig erfolgen. Das Schnorr-Identifikationsprotokoll weist zunächst lediglich die Kenntnis des zu einem öffentlichen Schlüssel gehörenden diskreten Logarithmus nach. Eine Aussage über die tatsächliche Identität einer Person ist nur möglich, wenn der öffentliche Schlüssel zuvor zuverlässig mit dieser Identität verknüpft wurde.

Die öffentlichen Parameter und der öffentliche Schlüssel müssen vor ihrer Verwendung auf Gültigkeit überprüft werden. Insbesondere muss sichergestellt werden, dass y ein zulässiges Element der vorgesehenen Untergruppe ist.

3.2 Ablauf des Drei-Schritt-Protokolls

Das Schnorr-Identifikationsprotokoll besteht aus drei aufeinanderfolgenden Nachrichten:

1. Commitment des Provers,
2. Challenge des Verifiers,

3. Response des Provers.

Ein Protokoll mit dieser Struktur und den später beschriebenen Sicherheitseigenschaften wird als Sigma-Protokoll (Σ-Protokoll) bezeichnet.

Zu Beginn der Protokollausführung kennen beide Parteien die öffentlichen Parameter und den öffentlichen Schlüssel y. Der geheime Schlüssel x ist ausschließlich dem Prover bekannt.

Schritt 1: Commitment

Der Prover wählt zunächst einen frischen Zufallswert r gleichverteilt aus $\mathbb{Z}_q$:

$$r \xleftarrow{\$} \mathbb{Z}_q.$$

Dieser temporäre Zufallswert wird auch als Nonce bezeichnet. Anschließend berechnet der Prover das Commitment

$$t = g^r.$$

Für die Realisierung in $\mathbb{Z}_p^*$ gilt entsprechend

$$t = g^r \bmod p.$$

Der Prover sendet ausschließlich das Commitment t an den Verifier. Der zugrunde liegende Zufallswert r wird nicht übertragen und muss geheim bleiben.

Das Commitment legt den Prover auf einen bestimmten Zufallswert fest, bevor ihm die Challenge des Verifiers bekannt ist.

Schritt 2: Challenge

Nach Erhalt des Commitments überprüft der Verifier dessen Gültigkeit und wählt anschließend eine Challenge c gleichverteilt aus $\mathbb{Z}_q$:

$$c \xleftarrow{\$} \mathbb{Z}_q.$$

Die Challenge wird an den Prover übertragen.

Entscheidend ist, dass die Challenge erst nach Erhalt des Commitments gewählt wird und für den Prover im Voraus nicht vorhersagbar ist.

Dadurch soll verhindert werden, dass ein Teilnehmer ohne Kenntnis des geheimen Schlüssels bereits vor Beginn der Prüfung ein passendes Transkript vorbereitet.

Schritt 3: Response

Nach Erhalt der Challenge berechnet der Prover unter Verwendung seines geheimen Schlüssels x und des zuvor gewählten Zufallswertes r die Response

$$s = (r + cx) \bmod q.$$

Anschließend sendet der Prover den Wert s an den Verifier.

Der geheime Schlüssel x sowie der temporäre Zufallswert r werden während des gesamten Protokollablaufs nicht übertragen.

Die drei Nachrichten lassen sich somit durch folgende Gleichungen zusammenfassen:

$$t = g^r \qquad \text{Commitment,} \tag{1}$$

$$c \xleftarrow{\$} \mathbb{Z}_q \qquad \text{Challenge,} \tag{2}$$

$$s = (r + cx) \bmod q \qquad \text{Response.} \tag{3}$$

Verifikation

Nach Erhalt der Response besitzt der Verifier sämtliche für die Prüfung benötigten öffentlichen Werte.

Er überprüft, ob die erhaltenen Werte gültig sind und ob folgende Gleichung erfüllt ist:

$$g^s \stackrel{?}{=} t\, y^c. \tag{4}$$

Im Fall einer multiplikativen Untergruppe von $\mathbb{Z}_p^*$ lautet die Prüfgleichung

$$g^s \stackrel{?}{\equiv} t\, y^c \pmod{p}.$$

Sind die empfangenen Werte gültig und ist die Gleichung erfüllt, akzeptiert der Verifier den Nachweis. Andernfalls wird die Authentifizierung abgelehnt.

Darstellung des Nachrichtenflusses

Der vollständige Nachrichtenfluss zwischen Prover und Verifier ist in Abbildung 1 dargestellt.

Prover		**Verifier**
Kenntnis von x		Kenntnis von $y = g^x$
$r \xleftarrow{\$} \mathbb{Z}_q$		
$t = g^r$	$\xrightarrow{t}$	
	$\xleftarrow{c}$	$c \xleftarrow{\$} \mathbb{Z}_q$
$s = (r + cx) \bmod q$	$\xrightarrow{s}$	
		$g^s \stackrel{?}{=} t\, y^c$

Abbildung 1: Nachrichtenfluss des Schnorr-Identifikationsprotokolls

Die während einer Protokollausführung übertragenen Nachrichten bilden gemeinsam das sogenannte Transkript

$$(t, c, s).$$

Ein Transkript wird als akzeptierend bezeichnet, wenn seine Werte gültig sind und die Prüfgleichung (4) erfüllt ist.

3.3 Korrektheit

Die Korrektheit des Schnorr-Identifikationsprotokolls ergibt sich aus den algebraischen Eigenschaften der verwendeten zyklischen Gruppe.

Es wird angenommen, dass beide Parteien das Protokoll ordnungsgemäß ausführen und der Prover den geheimen Schlüssel x kennt.

Aus der Definition des öffentlichen Schlüssels

$$y = g^x$$

sowie des Commitments

$$t = g^r$$

und der Response

$$s = (r + cx) \bmod q$$

folgt aufgrund der Gruppenordnung q:

$$\begin{aligned} g^s &= g^{r+cx} && (5)\\ &= g^r \cdot g^{cx} && (6)\\ &= g^r \cdot (g^x)^c && (7)\\ &= t \cdot y^c. && (8) \end{aligned}$$

Somit ist die Prüfgleichung

$$g^s = t \cdot y^c$$

für einen ehrlich erzeugten Protokollablauf erfüllt.

Die Reduktion der Response modulo q verändert das resultierende Gruppenelement nicht, da $g^q = e$ gilt.

Damit ist gezeigt, dass ein Prover, der den geheimen Schlüssel kennt und das Protokoll korrekt ausführt, die Prüfung eines ehrlichen Verifiers erfolgreich bestehen kann.

Diese Eigenschaft wird als Vollständigkeit (*Completeness*) bezeichnet.

3.4 Numerisches Beispiel

Zur Veranschaulichung des Protokollablaufs werden die bereits in Abschnitt 2 eingeführten Parameter verwendet:

$$p = 23, \qquad q = 11, \qquad g = 2.$$

Die Gruppe $\mathbb{G} = \langle 2 \rangle$ besitzt modulo 23 die Ordnung 11.

Schlüsselerzeugung. Der Prover wählt den geheimen Schlüssel

$$x = 7.$$

Der zugehörige öffentliche Schlüssel ergibt sich aus

$$\begin{aligned} y &= g^x \bmod p \\ &= 2^7 \bmod 23 \\ &= 13. \end{aligned}$$

Commitment. Für die Protokollausführung wird beispielhaft der Zufallswert

$$r = 4$$

gewählt.

Das Commitment lautet damit

$$\begin{aligned} t &= g^r \bmod p \\ &= 2^4 \bmod 23 \\ &= 16. \end{aligned}$$

Der Prover sendet den Wert $t = 16$ an den Verifier.

Challenge. Der Verifier wählt beispielhaft die Challenge

$$c = 3$$

und sendet diese an den Prover.

Response. Der Prover berechnet die Response

$$\begin{aligned} s &= (r + cx) \bmod q \\ &= (4 + 3 \cdot 7) \bmod 11 \\ &= 25 \bmod 11 \\ &= 3. \end{aligned}$$

Der berechnete Wert $s = 3$ wird an den Verifier übertragen.

Verifikation. Der Verifier überprüft die Gleichung

$$g^s \stackrel{?}{\equiv} t\, y^c \pmod{p}.$$

Für die linke Seite ergibt sich

$$g^s = 2^3 \bmod 23 = 8.$$

Die rechte Seite liefert

$$\begin{aligned} t\, y^c \bmod p &= 16 \cdot 13^3 \bmod 23 \\ &= 35152 \bmod 23 \\ &= 8. \end{aligned}$$

Da beide Seiten übereinstimmen, akzeptiert der Verifier den Nachweis:

$$8 = 8.$$

Die während der Protokollausführung verwendeten Werte sind in Tabelle 1 zusammengefasst.

Symbol	Wert	Bedeutung
p	23	Primzahl als Modulus
q	11	Gruppenordnung
g	2	Generator
x	7	Geheimer Schlüssel
y	13	Öffentlicher Schlüssel
r	4	Temporärer Zufallswert
t	16	Commitment
c	3	Challenge
s	3	Response

Tabelle 1: Beispiel einer erfolgreichen Schnorr-Identifikation

Das resultierende akzeptierende Transkript lautet

$$(t, c, s) = (16, 3, 3).$$

Der geheime Schlüssel x wurde während der Protokollausführung nicht direkt an den Verifier übertragen.

Aufgrund der bewusst kleinen Parameter könnte der geheime Schlüssel in diesem Beispiel allerdings durch Berechnung des diskreten Logarithmus leicht bestimmt werden. Das Beispiel dient daher ausschließlich der Veranschaulichung und stellt keine kryptografisch sichere Implementierung dar.

3.5 Sicherheitsidee und Grenzen

Das Schnorr-Identifikationsprotokoll besitzt drei wesentliche Eigenschaften, die für Sigma-Protokolle charakteristisch sind:

- Vollständigkeit (*Completeness*),
- spezielle Wissensextraktion (*Special Soundness*),
- Zero Knowledge gegenüber einem ehrlichen Verifier (*Honest-Verifier Zero Knowledge*).

Diese Eigenschaften beschreiben unterschiedliche Aspekte der Protokollsicherheit und müssen voneinander unterschieden werden.

Vollständigkeit

Die Vollständigkeit besagt, dass ein ehrlicher Prover, der den geheimen Schlüssel kennt, einen ehrlichen Verifier von dieser Kenntnis überzeugen kann.

Wie im Abschnitt zur Korrektheit gezeigt wurde, gilt für eine ordnungsgemäße Protokollausführung stets

$$g^s = t\, y^c.$$

Unter der Voraussetzung gültiger Parameter und einer korrekten Ausführung akzeptiert der Verifier daher den Nachweis mit Wahrscheinlichkeit 1.

Die Vollständigkeit ist somit eine Aussage über das Verhalten ehrlicher Teilnehmer und noch kein Nachweis dafür, dass sich ein Angreifer nicht erfolgreich authentifizieren kann.

Spezielle Wissensextraktion

Die spezielle Wissensextraktion beschreibt eine Eigenschaft, die für den Nachweis der Kenntnis des geheimen Schlüssels wesentlich ist.

Angenommen, es liegen zwei akzeptierende Transkripte

$$(t, c_1, s_1) \qquad \text{und} \qquad (t, c_2, s_2)$$

mit demselben Commitment t, jedoch unterschiedlichen Challenges

$$c_1 \neq c_2$$

vor.

Da beide Transkripte akzeptierend sind, gilt

$$g^{s_1} = t\, y^{c_1}, \tag{9}$$
$$g^{s_2} = t\, y^{c_2}. \tag{10}$$

Durch Multiplikation der ersten Gleichung mit dem Inversen der zweiten Gleichung ergibt sich

$$g^{s_1 - s_2} = y^{c_1 - c_2}.$$

Mit $y = g^x$ folgt

$$g^{s_1 - s_2} = g^{x(c_1 - c_2)}.$$

Da g die Ordnung q besitzt, müssen die Exponenten modulo q übereinstimmen:

$$s_1 - s_2 \equiv x(c_1 - c_2) \pmod{q}.$$

Weil q prim ist und $c_1 \neq c_2$ gilt, besitzt $c_1 - c_2$ ein multiplikatives Inverses modulo q.

Durch Multiplikation mit diesem Inversen erhält man

$$x = (s_1 - s_2)(c_1 - c_2)^{-1} \bmod q. \tag{11}$$

Aus zwei akzeptierenden Transkripten mit identischem Commitment und unterschiedlichen Challenges kann somit der geheime Schlüssel effizient berechnet werden.

Diese Eigenschaft wird in einem theoretischen Sicherheitsbeweis durch einen sogenannten Wissensextraktor genutzt.

Der Wissensextraktor betrachtet eine Protokollausführung und versetzt den untersuchten Prover gedanklich in den Zustand unmittelbar nach dem Senden des Commitments zurück. Anschließend wird eine andere Challenge verwendet.

Kann der Prover mit hinreichend hoher Wahrscheinlichkeit gültige Antworten auf unterschiedliche Challenges für dasselbe Commitment erzeugen, lässt sich daraus der zum öffentlichen Schlüssel gehörende diskrete Logarithmus extrahieren.

Dieses Verfahren wird als *Rewinding* bezeichnet. Es handelt sich um eine Technik des Sicherheitsbeweises und nicht um einen regulären Schritt des eigentlichen Identifikationsprotokolls.

Die spezielle Wissensextraktion bildet damit eine Grundlage für die Interpretation des Schnorr-Protokolls als Wissensnachweis.

Für eine gleichverteilt aus $\mathbb{Z}_q$ gewählte Challenge beträgt der sogenannte Wissensfehler des Sigma-Protokolls

$$\varepsilon = \frac{1}{q}.$$

Ein Teilnehmer kann beispielsweise im Voraus eine Challenge erraten und ein passendes Transkript konstruieren. Unter der Annahme einer unabhängigen und gleichverteilten Challenge beträgt die Erfolgswahrscheinlichkeit einer solchen Strategie genau $1/q$.

Ein Prover, dessen Erfolgswahrscheinlichkeit wesentlich über diesem Wissensfehler liegt, kann unter den Voraussetzungen des entsprechenden Extraktionsarguments zur Berechnung eines gültigen geheimen Schlüssels verwendet werden.

Bei ausreichend großer Gruppenordnung und unter der Annahme der Schwierigkeit des diskreten Logarithmusproblems ergibt sich daraus die Sicherheitsidee des Identifikationsverfahrens.

Honest-Verifier Zero Knowledge

Die Zero-Knowledge-Eigenschaft beschreibt, dass ein Nachweis keine zusätzlichen Informationen über das geheime Wissen des Provers offenlegen soll.

Für das Schnorr-Identifikationsprotokoll lässt sich diese Eigenschaft zunächst für einen ehrlichen Verifier zeigen.

Ein ehrlicher Verifier hält sich an die Protokollspezifikation und wählt seine Challenge insbesondere unabhängig und gleichverteilt aus dem vorgesehenen Wertebereich.

Die Beweisidee besteht darin, einen Simulator zu konstruieren, der ein akzeptierendes Transkript erzeugen kann, ohne den geheimen Schlüssel x zu kennen.

Der Simulator kennt lediglich die öffentlichen Parameter und den öffentlichen Schlüssel y.

Er führt folgende Schritte aus:

1. Wähle eine Challenge c gleichverteilt aus $\mathbb{Z}_q$:

$$c \xleftarrow{\$} \mathbb{Z}_q.$$

2. Wähle eine Response s unabhängig und gleichverteilt aus $\mathbb{Z}_q$:

$$s \xleftarrow{\$} \mathbb{Z}_q.$$

3. Berechne das Commitment rückwärts aus der Prüfgleichung:

$$t = g^s \cdot y^{-c}.$$

Der Ausdruck y^{-c} bezeichnet dabei die entsprechende inverse Potenz des Gruppenelements y.
Der Simulator gibt anschließend das Transkript

$$(t, c, s)$$

aus.

Dieses Transkript erfüllt die Prüfgleichung, denn

$$\begin{aligned} t\, y^c &= (g^s \cdot y^{-c}) \cdot y^c \\ &= g^s. \end{aligned}$$

Es handelt sich somit um ein akzeptierendes Transkript, obwohl der Simulator den geheimen Schlüssel nicht kennt.

Darüber hinaus besitzen die simulierten Transkripte dieselbe Wahrscheinlichkeitsverteilung wie die Transkripte einer ehrlichen Protokollausführung.

In einer regulären Protokollausführung gilt für jede fest gewählte Challenge c

$$s = (r + cx) \bmod q.$$

Da r gleichverteilt aus $\mathbb{Z}_q$ gewählt wird, ist auch s gleichverteilt in $\mathbb{Z}_q$. Die Addition des festen Wertes cx verändert diese Gleichverteilung nicht.

Das zugehörige Commitment lässt sich durch

$$t = g^r = g^{s-cx} = g^s \cdot y^{-c}$$

darstellen.

Damit stimmen die Verteilungen der simulierten und der ehrlich erzeugten Transkripte exakt überein.

Dies wird als perfekte Honest-Verifier-Zero-Knowledge- Eigenschaft bezeichnet.

Der Verifier erhält aus einer ehrlichen Protokollausführung folglich keine zusätzliche Information über den geheimen Schlüssel, die nicht auch aus den öffentlichen Informationen simuliert werden könnte.

Dabei ist zu beachten, dass der öffentliche Schlüssel selbst bereits den geheimen Schlüssel mathematisch festlegt. Die praktische Geheimhaltung des Schlüssels beruht weiterhin auf der Schwierigkeit des diskreten Logarithmusproblems.

Abgrenzung zur Wissenseigenschaft. Die Möglichkeit, ein akzeptierendes Transkript ohne Kenntnis des geheimen Schlüssels zu simulieren, steht nicht im Widerspruch zur Wissenseigenschaft des Protokolls.

Der Simulator darf zunächst die Challenge und die Response auswählen und erst anschließend das passende Commitment berechnen.

Während einer tatsächlichen Protokollausführung muss der Prover das Commitment jedoch bereits übertragen, bevor die Challenge des Verifiers bekannt ist.

Die zeitliche Reihenfolge der Nachrichten ist daher ein wesentlicher Bestandteil der Protokollsicherheit.

Ein einzelnes akzeptierendes Transkript stellt deshalb für sich allein keinen unabhängig überprüfbaren Beweis dar, dass eine bestimmte Person tatsächlich den geheimen Schlüssel kennt.

Bedeutung frischer Zufallswerte

Die Sicherheit des Schnorr-Identifikationsprotokolls setzt voraus, dass für jede Protokollausführung ein neuer, unabhängiger und kryptografisch sicherer Zufallswert r erzeugt wird.

Wird derselbe Zufallswert in zwei Ausführungen verwendet, so entsteht dasselbe Commitment

$$t = g^r.$$

Erhält der Prover dabei zwei unterschiedliche Challenges c_1 und c_2, ergeben sich die Responses

$$s_1 = (r + c_1 x) \bmod q, \tag{12}$$
$$s_2 = (r + c_2 x) \bmod q. \tag{13}$$

Durch Subtraktion der beiden Gleichungen fällt der Zufallswert r weg:

$$s_1 - s_2 \equiv (c_1 - c_2)x \pmod{q}.$$

Der geheime Schlüssel kann daher unmittelbar mithilfe von Gleichung (11) berechnet werden.

Die Wiederverwendung eines Zufallswertes kann somit zur vollständigen Kompromittierung des langfristigen geheimen Schlüssels führen.

Numerisches Beispiel einer Nonce-Wiederverwendung. Es werden erneut die Beispielparameter

$$p = 23, \qquad q = 11, \qquad g = 2, \qquad x = 7$$

verwendet.

Angenommen, der Prover verwendet in zwei verschiedenen Protokollausführungen denselben Zufallswert

$$r = 4.$$

Das Commitment lautet in beiden Ausführungen

$$t = 2^4 \bmod 23 = 16.$$

Für die erste Challenge $c_1 = 3$ erhält man

$$\begin{aligned} s_1 &= (4 + 3 \cdot 7) \bmod 11 \\ &= 3. \end{aligned}$$

Für die zweite Challenge $c_2 = 5$ ergibt sich

$$\begin{aligned} s_2 &= (4 + 5 \cdot 7) \bmod 11 \\ &= 6. \end{aligned}$$

Damit liegen die beiden akzeptierenden Transkripte

$$(16, 3, 3) \qquad \text{und} \qquad (16, 5, 6)$$

vor.

Der geheime Schlüssel kann nun durch

$$\begin{aligned} x &= (s_1 - s_2)(c_1 - c_2)^{-1} \bmod q \\ &= (3 - 6)(3 - 5)^{-1} \bmod 11 \\ &= (-3)(-2)^{-1} \bmod 11 \end{aligned}$$

berechnet werden.

Da

$$(-2)^{-1} \equiv 5 \pmod{11}$$

gilt, ergibt sich

$$\begin{aligned} x &= (-3) \cdot 5 \bmod 11 \\ &= -15 \bmod 11 \\ &= 7. \end{aligned}$$

Der ursprüngliche geheime Schlüssel wurde damit vollständig rekonstruiert.

Dieses Beispiel verdeutlicht, dass die Sicherheit des Protokolls nicht ausschließlich von der Schwierigkeit des diskreten Logarithmusproblems abhängt, sondern auch von der Qualität und korrekten Verwendung der erzeugten Zufallswerte.

Auch die Offenlegung oder Vorhersagbarkeit eines einzelnen Zufallswertes kann den geheimen Schlüssel gefährden. Für eine von null verschiedene Challenge c gilt bei bekanntem r

$$x = (s - r)c^{-1} \bmod q.$$

Der temporäre Zufallswert muss daher sowohl während als auch nach seiner Verwendung vor unbefugtem Zugriff geschützt werden. Insbesondere darf ein bereits verwendetes Commitment nicht für die Beantwortung weiterer Challenges wiederverwendet werden.

Angreifermodell und weitere Sicherheitsgrenzen

Die beschriebenen mathematischen Eigenschaften gelten unter bestimmten Voraussetzungen. Sie erlauben keine pauschale Aussage über die Sicherheit jeder praktischen Implementierung.

Insbesondere sind folgende Aspekte zu berücksichtigen:

Unvorhersehbare Challenges: Die Challenge muss unabhängig vom Prover und mithilfe eines kryptografisch sicheren Zufallszahlengenerators erzeugt werden. Ist die Challenge vorhersehbar, kann ein Angreifer ein passendes Commitment vorbereiten, ohne den geheimen Schlüssel zu kennen.

Validierung der Gruppenparameter: Die verwendeten Parameter müssen die notwendigen mathematischen Voraussetzungen erfüllen. Dazu zählen insbesondere die Primzahleigenschaften von p und q, die Teilbarkeitsbedingung $q \mid (p-1)$ sowie die korrekte Ordnung des Generators g.

Validierung der Gruppenelemente: Öffentliche Schlüssel und Commitments müssen auf ihre Zugehörigkeit zur vorgesehenen Untergruppe überprüft werden.

Für ein Element u in der multiplikativen Untergruppe müssen insbesondere

$$1 \leq u \leq p - 1$$

und

$$u^q \equiv 1 \pmod{p}$$

gelten.

Für den öffentlichen Schlüssel ist zusätzlich $y \neq 1$ erforderlich, da der zugehörige diskrete Logarithmus andernfalls öffentlich bekannt wäre.

Replay-Angriffe: Ein aufgezeichnetes Transkript kann nicht ohne Weiteres für eine spätere Authentifizierung verwendet werden, da der Verifier eine neue, unabhängige Challenge erzeugt.

Stimmt die neue Challenge mit der aufgezeichneten Challenge überein, ist ein Replay jedoch grundsätzlich möglich. Bei gleichverteilter Auswahl aus $\mathbb{Z}_q$ beträgt die Wahrscheinlichkeit hierfür $1/q$. Die Challenge muss daher einen ausreichend großen Wertebereich besitzen.

Relay-Angriffe: Eine frische Challenge verhindert nicht zwangsläufig die Weiterleitung einer laufenden Authentifizierung. Ein Angreifer könnte Nachrichten zwischen einem legitimen Prover und einem Verifier weiterleiten, ohne den geheimen Schlüssel selbst zu kennen.

Zusätzliche Mechanismen zur Bindung der Authentifizierung an den Kommunikationskanal, die Gegenstelle oder den Anwendungskontext können daher erforderlich sein.

Manipulierte Verifier: Die hier gezeigte Zero-Knowledge-Eigenschaft bezieht sich auf einen ehrlichen Verifier. Daraus folgt nicht unmittelbar eine entsprechende Sicherheit gegenüber beliebig vom Protokoll abweichenden Verifiern.

Weitergehende Aussagen erfordern ein präzises Sicherheitsmodell und entsprechende Beweise.

Implementierungssicherheit: Fehler bei der Implementierung, unsichere Zufallszahlengeneratoren, Seitenkanalangriffe oder ein unzureichender Schutz des geheimen Schlüssels können die Sicherheit des Verfahrens beeinträchtigen.

Quantencomputer: Die langfristige Sicherheit des Verfahrens hängt von der Schwierigkeit des diskreten Logarithmusproblems ab. Ein hinreichend leistungsfähiger, fehlertoleranter Quantencomputer könnte dieses Problem mithilfe des Shor-Algorithmus effizient lösen.

Das klassische Schnorr-Identifikationsprotokoll ist daher kein postquantenkryptografisches Verfahren.

3.6 Abgrenzung zu nichtinteraktiven Verfahren

Das in diesem Abschnitt beschriebene Schnorr-Identifikationsprotokoll setzt eine aktive Interaktion zwischen Prover und Verifier voraus.

Eine bekannte Erweiterung besteht darin, die zufällige Challenge des Verifiers durch einen aus öffentlichen Informationen berechneten kryptografischen Hashwert zu ersetzen. Dieser Ansatz wird als Fiat-Shamir-Transformation bezeichnet.

Abstrakt kann eine solche Challenge beispielsweise durch

$$c = H(g, y, t, \text{Kontext}) \bmod q$$

dargestellt werden.

Der Prover kann dadurch die Challenge selbst berechnen, ohne zuvor eine Nachricht des Verifiers erhalten zu müssen.

Die konkrete Definition und Kodierung der Hash-Eingaben sowie die Abbildung des Hashwertes in den vorgesehenen Challenge-Bereich sind dabei sicherheitsrelevant.

Diese Transformation ermöglicht unter geeigneten Annahmen die Konstruktion nichtinteraktiver Wissensnachweise und bildet außerdem eine Grundlage für Schnorr-Signaturverfahren.

Bei einer digitalen Signatur werden zusätzlich die zu signierende Nachricht und gegebenenfalls weitere Kontextinformationen in die Berechnung der Challenge einbezogen.

Die Sicherheit einer solchen Erweiterung muss jedoch gesondert untersucht werden. Die Eigenschaften des interaktiven Protokolls können nicht ohne weitere Annahmen unmittelbar auf jede nichtinteraktive Variante übertragen werden.

3.7 Zusammenfassung

Das Schnorr-Identifikationsprotokoll ermöglicht den Nachweis der Kenntnis eines geheimen diskreten Logarithmus, ohne diesen direkt an den Verifier zu übertragen.

Die Kombination aus Commitment, Challenge und Response gewährleistet bei einer korrekten Protokollausführung die Vollständigkeit des Verfahrens.

Die spezielle Wissensextraktion zeigt, dass aus zwei akzeptierenden Transkripten mit identischem Commitment und unterschiedlichen Challenges der geheime Schlüssel berechnet werden kann.

Gleichzeitig lässt sich durch Simulation nachweisen, dass eine ehrliche Protokollausführung keine zusätzlichen Informationen über den geheimen Schlüssel offenlegt.

Die Sicherheit des Verfahrens beruht somit auf dem Zusammenspiel der mathematischen Eigenschaften des Protokolls, der Schwierigkeit des diskreten Logarithmusproblems und der korrekten praktischen Implementierung.

4 Praxisbezug und Aufwandsabschätzung

4.1 Optional: Ein kleines Rechenbeispiel

Schreibhinweis: Ein vollständiges, ausdrücklich unsicheres Lernbeispiel durchrechnen: etwa $p = 23$, $q = 11$, $g = 2$, $x = 3$, $r = 4$ und $c = 5$. Öffentlichen Schlüssel, Commitment, Response und Prüfung selbst berechnen. Kleine Zahlen dienen ausschließlich der Nachvollziehbarkeit.

4.2 Aufwand eines Angriffs

Schreibhinweis: Das Angriffsziel festlegen: Schlüsselrückgewinnung durch diskreten Logarithmus oder erfolgreiche Täuschung in einer Protokollrunde. Brute Force und generische Verfahren wie Baby-Step–Giant-Step bzw. Pollard-Rho anhand belastbarer Quellen vergleichen: Gruppenoperationen, Speicherbedarf und Abhängigkeit von q angeben.

Schreibhinweis: Bei einer Zeitabschätzung die angenommene Rate der Gruppenoperationen explizit nennen. Für Untergruppen endlicher Körper auch spezialisierte Angriffe berücksichtigen; aus q allein folgt dort kein Sicherheitsniveau. Das Lernbeispiel klar von realistischen Parametern trennen.

4.3 Anwendungsfälle und verwandte Verfahren

Schreibhinweis: Quellenbelegte Anwendungsfälle recherchieren. Zwischen direkter interaktiver Identifikation und darauf aufbauenden Verfahren unterscheiden. Die nichtinteraktive Variante und Schnorr-Signaturen nur als Ausblick behandeln. RFC 8235 beschreibt die nichtinteraktive Variante: [2].

5 Vor- und Nachteile sowie Einsatzbedingungen

5.1 Vorteile

Schreibhinweis: Unter anderem Rechen- und Kommunikationsaufwand sowie Nachweis ohne Offenlegung des Geheimnisses bewerten. Jede Aussage begründen oder belegen.

5.2 Nachteile und Voraussetzungen

Schreibhinweis: Interaktion, Zufallszahlenerzeugung, Schlüsselzuordnung, sichere Implementierung und Grenzen der zugrunde liegenden Annahmen besprechen.

5.3 Wann lohnt sich das Verfahren?

Schreibhinweis: Aus den vorherigen Punkten konkrete Einsatzbedingungen ableiten. Bezug zur Problemstellung herstellen und Vergleichskriterien offenlegen.

6 Fazit

Schreibhinweis: Leitfrage beantworten, wichtigste Erkenntnisse zusammenführen und Grenzen des Papers benennen. Optional kurzer Ausblick. Keine neuen Beweise oder unvorbereiteten technischen Begriffe einführen.

Literaturverzeichnis

[1] Claus-Peter Schnorr. „Efficient Signature Generation by Smart Cards“. In: *Journal of Cryptology* 4 (1991), S. 161–174. DOI: 10.1007/BF00196725.

[2] Feng Hao. *Schnorr Non-interactive Zero-Knowledge Proof*. RFC 8235. RFC Editor, Sep. 2017. DOI: 10.17487/RFC8235. URL: https://www.rfc-editor.org/rfc/rfc8235 (besucht am 13. 11. 2026).

[3] Wikipedia. *Schnorr-Identifikation*. Einstiegsquelle; technische Aussagen mit Fachliteratur abgleichen. URL: https://de.wikipedia.org/wiki/Schnorr-Identifikation (besucht am 13. 11. 2026).